SECURE STEGANOGRAPHY SOFTWARE SUITE RESILIENT TO STEGANALYSIS

BY

OGUNDIRAN TEMILOLUWA SAMUEL

(22CG031905)

A PROJECT SUBMITTED TO THE DEPARTMENT OF COMPUTER AND INFORMATION SCIENCES, COLLEGE OF SCIENCE AND TECHNOLOGY, COVENANT UNIVERSITY OTA, OGUN STATE.

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE BACHELOR OF SCIENCE (HONOURS) DEGREE IN COMPUTER SCIENCE.

DECEMBER 2025

CERTIFICATION

I hereby certify that this project was conducted by Ogundiran Temiloluwa Samuel in the Department of Computer and Information Sciences, College of Science and Technology, Covenant University, Ogun State, Nigeria, under my supervision.

Miss Ti-Jesu Tunde-Adeleke _____

Supervisor Signature and Date

Prof. Aderonke A Oni _____

Head of Department Signature and Date

ACKNOWLEDGEMENTS

First and foremost, God is the source of my immense gratitude, as He enabled this work to be completed. I also want to thank Covenant University and my family for their unwavering provision and support.

My acknowledgement to the Head of the Department of Computer and Information Science Prof Aderonke Oni, for her guidance, I am thankful to Miss Ti-Jesu Tunde-Adeleke, my supervisor, for her direction and oversight in this project.

TABLE OF CONTENT

TOC \o "1-3" \h \z \u CERTIFICATION 2

ACKNOWLEDGEMENTS 3

TABLE OF CONTENT 4

LIST OF FIGURES **Error! Bookmark not defined.**

LIST OF TABLES **Error! Bookmark not defined.**

ABBREVIATIONS **Error! Bookmark not defined.**

CHAPTER ONE

INTRODUCTION

Background Information

The growth of digital communication has made information exchange faster, wider, and more exposed than at any previous time. Sensitive personal, institutional, and commercial messages now move across networks where interception is realistic, and this has made information security a central concern in computer science and cybersecurity.

Traditional protection of digital communication has relied mainly on cryptography. Cryptography secures the content of a message by transforming readable plaintext into ciphertext that cannot be interpreted without the proper key. This gives strong confidentiality, but it does not hide the fact that a secret communication is taking place. In situations where the mere presence of encrypted content may attract attention, cryptography alone is not always sufficient.

Steganography addresses this second problem by concealing the existence of a message inside an apparently innocent carrier such as an image, audio signal, or video stream. In image steganography, the sender embeds secret information into a cover image and produces a stego image that appears visually similar to the original. The receiver then extracts the hidden information from that stego image using an agreed procedure. A strong steganographic system therefore aims at four closely related goals: imperceptibility, payload capacity, robustness, and resistance to detection.

Classical image steganography methods such as Least Significant Bit substitution and transform-domain embedding made important early contributions to covert communication. However, these

methods typically follow fixed embedding rules. Once those rules introduce measurable statistical disturbances into the cover image, modern steganalysis systems can learn to detect them. What used to be a hidden message under simple visual inspection can now be exposed by data-driven classifiers that detect minute embedding artifacts.

The introduction of deep learning has changed both sides of this contest. On one side, deep neural networks have strengthened steganalysis by learning subtle residual patterns that handcrafted feature engineering could not reliably capture. On the other side, adversarial-network-based steganography has provided a more adaptive way of embedding information. Instead of manually deciding where and how to place message bits, an adversarial model learns embedding behaviour that seeks to remain close to natural image statistics while still preserving decodability.

This project is motivated by the fact that many publicly accessible steganography tools are still built around older embedding strategies and do not adequately address present-day steganalysis threats. In addition, practical secure software in this area remains limited. Some research works propose stronger adversarial or deep-learning-based steganographic methods, but these are often presented as technical prototypes rather than complete software suites. As a result, there is a gap between academic advances in steganography and real systems that ordinary users can operate securely through a clear encode, decode, and verification workflow.

For this reason, this project presents an original secure steganography software suite that uses an adversarial-network-based image steganography method as its embedding backbone and strengthens it with passphrase-protected payload handling, structured encode-decode-check workflows, and a user-facing application interface. The aim is not only to hide information in images, but to do so through a software system that is practical, secure, and more resilient to modern steganalysis than conventional methods.

Statement of Problem

The main problem addressed in this study is that many conventional steganography tools are not sufficiently resilient to present-day steganalysis. Spatial-domain and transform-domain methods often introduce statistical patterns that can be detected by machine-learning-based steganalysis systems, especially when payload size increases or when the attacker has access to modern residual-network detectors.

A second problem is that practical secure software in this area remains limited. Some research works propose stronger adversarial or deep-learning-based steganographic methods, but these are often presented as technical prototypes rather than complete software suites. As a result, there is a gap between model-level progress and software systems that ordinary users can use securely through a controlled encode, decode, and verification workflow.

Aim and Objectives

The aim of this project is to design, implement, and evaluate a secure steganography software suite that conceals text messages inside images using an adversarial-network-based approach and improves practical security through controlled, passphrase-protected recovery.

To achieve this aim, the study pursues the following objectives:

- To review the major concepts, methods, tools, and research trends in image steganography, steganalysis, and adversarial-network-based information hiding.
- To analyze the limitations of conventional steganography methods and identify the gaps that justify a more secure and practical software suite.
- To design a complete steganography system that supports image encoding, decoding, hidden-data checking, and passphrase-protected message recovery.
- To implement the proposed system using a web-based software architecture with a deep-learning steganography backbone and a secure payload protection layer.
- To evaluate the system using established image-quality and security criteria and to compare the adopted adversarial method with less effective traditional approaches.

Methodology

This study adopts a structured development methodology in which the work proceeds from problem definition to design, implementation, and evaluation. The process is consistent with the logic of the Waterfall model because the major phases of the project were clearly defined from the outset, but each phase was also grounded in technical validation and iterative refinement of details within the phase itself.

The first phase was the literature review and problem analysis stage. In this stage, relevant concepts in steganography, steganalysis, cryptography, and adversarial learning were reviewed.

Existing classical tools and modern research methods were compared in order to identify the most important practical and research gaps.

The second phase was system analysis and design. Here, the requirements of the proposed suite were defined, the operational modules were identified, and the system architecture was specified. Particular attention was given to the encode, decode, and check workflow and to the introduction of a passphrase-protected secure payload layer.

The third phase was implementation. The system was implemented as a web-based software suite with a frontend interface, a FastAPI backend, a secure payload service layer, and an adversarial-network-based steganography engine. Supporting functions such as upload normalization, temporary output management, and secure message handling were also developed.

The fourth phase was testing and evaluation. This involved functional verification of the software modules, security-oriented testing of passphrase-controlled recovery, and technical evaluation using image-quality and detection-related measures discussed in the literature. The project also compared the adopted adversarial approach against conventional steganographic methods such as LSB-based and transform-domain techniques.

Significance of the Study

This study is significant because it contributes to both the academic and practical sides of secure communication.

Practical significance: the project provides a usable secure steganography suite rather than stopping at model-level theory. This makes stronger information-hiding techniques more accessible to users who need a clear encode, decode, and verification workflow.

Security significance: the system combines concealed communication with passphrase-protected payload handling, thereby adding a second layer of protection even when a hidden payload is recovered from an image.

Academic significance: the work contributes to the continuing contest between steganography and steganalysis by examining how adversarial-network-based methods compare with weaker traditional approaches in terms of security, usability, and detectability.

Chapter Outline

Chapter One introduces the study, states the problem, presents the aim and objectives, explains the methodology, and outlines the significance of the work. Chapter Two reviews the literature on classical steganography, steganalysis, machine learning, adversarial information hiding, and practical software tools, and then identifies the research gap addressed by the proposed suite. Chapter Three presents the system analysis and design of the secure steganography software suite, including the architecture, requirements, security design, datasets, and evaluation metrics. Chapter Four discusses the implementation of the system and its evaluation, including a comparison between the adopted adversarial method and conventional approaches. Chapter Five presents the summary, recommendations, and conclusion of the study.

CHAPTER TWO

LITERATURE REVIEW

2.1 Preamble

This chapter reviews the major theories, methods, systems, and tools relevant to the design of the proposed secure steganography software suite. It begins with the broad concepts that define the field, then examines previous works in classical and adversarial steganography, reviews practical software tools, and identifies the gaps that motivate the present study.

2.2 Review of Relevant Concepts

This section reviews the major concepts on which the proposed suite depends.

2.2.1 Cryptography

Cryptography is the discipline of securing information by transforming plaintext into an unreadable form that can only be reversed by an authorized party. In practical secure communication, cryptography is the main mechanism for preserving confidentiality of content.

In the context of this project, cryptography complements steganography rather than replacing it. The software suite protects the hidden message with passphrase-based authenticated encryption before the payload is embedded in an image. This means the project secures both the existence of the communication and the meaning of the communication.

2.2.2 Steganography

Steganography is the art and science of concealing information inside another medium in a way that hides the presence of the communication itself. In image steganography, the carrier is a digital image and the hidden information may be text, binary data, or another media object.

A sound steganographic system should satisfy imperceptibility, capacity, robustness, and secrecy. These four requirements are often in tension, which makes steganography a problem of careful trade-offs rather than a single optimization target.

2.2.3 Spatial Domain Steganography

Spatial domain steganography hides data by modifying pixel values directly. The most common example is Least Significant Bit substitution, in which message bits are written into the least

significant positions of pixel intensities. The attraction of this family is its simplicity and high raw capacity.

The main weakness of spatial methods is that their statistical footprints can often be detected by classical and deep learning steganalysis. Even when the visual distortion is small, the hidden modifications may disturb natural image statistics enough to reveal the presence of embedded content.

2.2.4 Transform Domain Steganography

Transform domain methods embed data in frequency-related representations rather than in raw pixel values. JPEG-oriented techniques based on the Discrete Cosine Transform are common examples. These methods often provide better resilience than simple spatial-domain methods because the embedding is distributed across transformed coefficients.

Although transform-domain techniques improve on simple LSB substitution, they are still not immune to advanced steganalysis. Once a detector learns the coefficient-level disturbances introduced by embedding, their security advantage can decline significantly.

2.2.5 Steganalysis

Steganalysis is the process of identifying whether hidden data exists in a carrier medium. Earlier steganalysis approaches depended on handcrafted statistical features and targeted attacks. Modern approaches increasingly rely on machine learning and deep residual representations to discover more subtle embedding traces.

The strength of present-day steganalysis is the core reason why stronger steganography systems are needed. A practical hidden communication system can no longer assume that low visual distortion alone is enough to guarantee secrecy.

2.2.6 Machine Learning

Machine learning refers to computational methods that learn patterns from data instead of relying only on explicitly written rules. In image security applications, this makes it possible to learn subtle patterns that are difficult to capture using manual statistical engineering alone.

2.2.7 Machine Learning in Steganalysis

When machine learning is applied to steganalysis, the detection problem becomes a classification task. The model learns to separate cover images from stego images by studying the residual disturbances introduced during embedding. This has substantially improved detection performance compared with older handcrafted techniques.

Deep learning steganalysis models are especially dangerous to classical embedding schemes because they can learn non-linear patterns distributed across the image. This creates pressure for steganography systems to adopt more adaptive and data-driven concealment strategies.

2.2.8 Generative Adversarial Networks

Generative Adversarial Networks are learning systems in which one model learns to generate outputs while another model learns to distinguish generated outputs from genuine ones. The adversarial pressure created by this competition has made GANs effective in several image-generation tasks.

For steganography, this adversarial principle is useful because the embedding model can be trained to reduce detectable artifacts while preserving message recovery. This makes GAN-based steganography a promising response to the stronger detection capabilities of deep learning steganalysis.

2.3 Review of Existing Works

This section reviews the principal streams of prior work that shaped the design of the proposed system. The first stream focuses on steganalysis, especially the movement from handcrafted statistical detection to machine-learning-driven detection. The second stream focuses on modern generative approaches to steganography that aim to reduce detectability by learning adaptive embedding strategies.

2.3.1 Machine Learning Steganalysis Research

Early steganalysis techniques depended on explicit statistical tests tailored to known embedding patterns. Over time, these methods were extended into universal steganalysis frameworks that used richer feature sets and conventional classifiers such as Support Vector Machines. Although effective, these systems still depended heavily on handcrafted feature design.

The modern era of steganalysis is increasingly dominated by deep learning. Convolutional neural networks and residual-network-based detectors learn high-dimensional residual patterns directly from data and are therefore more difficult for fixed-rule embedding methods to evade. This development is one of the key reasons why conventional steganographic methods are no longer sufficient in high-threat environments.

2.3.2 GAN-Based Steganography Research

Adversarial-network-based steganography emerged as a response to stronger steganalysis. Instead of relying on manually defined embedding heuristics, these methods learn to generate stego images whose statistical properties remain close to those of natural images while still enabling message recovery. This introduces a more adaptive and data-driven form of information hiding.

A major reference point in this area is the work of Zhang et al. (2019), which presented a high-capacity image steganography model based on adversarial training and dense connectivity. Their results showed that deep-learning-based steganography could achieve stronger payload and image-quality trade-offs than earlier approaches, while also reducing detectability under several steganalysis settings. This line of work provides the technical foundation that informs the present project, although the present project is positioned as an original secure software suite rather than a reproduction of that implementation.

2.4 Review of Existing Systems

Existing systems in this area fall into two broad groups. The first group contains classical public tools such as OpenStego and Steghide. These tools are practical and easy to use, but they largely depend on older embedding strategies that are less resilient to machine-learning-based steganalysis. The second group contains research-oriented adversarial or deep-learning steganography prototypes, which often show stronger security characteristics but do not always provide complete user-facing software workflows.

The proposed project sits between these two groups. It takes the stronger embedding philosophy of adversarial-network-based steganography and combines it with software engineering features expected of a usable suite: structured encode, decode, and check workflows, passphrase-protected payload handling, and web-based accessibility.

2.5 Review of Existing Methods

Classical distortion-cost methods aim to minimize the observable changes introduced during embedding by assigning different costs to different image locations. Such methods are useful because they formalize the trade-off between payload and detectability, but they still operate within fixed handcrafted assumptions about the cover image. Once steganalysis models learn the residual artifacts associated with those assumptions, their secrecy degrades.

Adversarial training strategies move beyond fixed cost rules by allowing a generator-like model to learn embedding behaviour against a critic or detector. In the context of steganography, this means the embedding engine is optimized not only for message recovery and image similarity, but also for reduced detectability. This is a stronger design principle for modern secure communication.

Imperceptibility is typically evaluated through quantitative image-quality measures. In practical terms, the most widely used metrics include Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and Structural Similarity Index Measure (SSIM). These metrics do not fully replace human judgement, but they provide repeatable and comparable evidence for the quality of stego images.

2.6 Review of Existing Tools

OpenStego is one of the widely accessible open-source steganography tools. It provides a graphical interface and allows message hiding in image files with optional encryption support. However, its main embedding logic remains rooted in classical techniques, which limits its resilience against stronger detection methods.

Steghide is another commonly referenced tool that supports image and audio carriers and provides password-protected extraction. Compared with basic LSB tools, it benefits from a more structured embedding process and compression support, but it still belongs to an earlier generation of practical steganography software and does not fundamentally solve the challenge posed by modern deep steganalysis.

These tools are still useful as practical baselines because they are accessible, understandable, and widely cited. Their weakness under stronger steganalysis is precisely what creates the need for a more secure software suite built around a modern adversarial embedding strategy.

2.7 Research Gaps

Several research and implementation gaps emerge from the literature and tool review:

Gap 1: There is a shortage of practical secure steganography software that combines modern adversarial embedding with a complete end-user workflow.

Gap 2: Many user-friendly public tools still depend on older embedding methods that are relatively weak against contemporary steganalysis.

Gap 3: Software-level security features such as controlled payload recovery and passphrase-based protection are not consistently integrated into modern research prototypes.

Gap 4: Comparative discussion often separates model-level performance from actual deployable software concerns such as usability, validation, and secure handling of outputs.

The present study addresses these gaps by designing a secure software suite that combines an adversarial-network-based embedding core with a practical web interface, encrypted payload handling, and a verification-oriented workflow.

2.8 Proposed System

The proposed system is a secure image steganography software suite that allows a user to encode a text message into an image, decode a hidden message with the correct passphrase, and check whether a candidate image contains a valid protected payload. The system is designed around a dense adversarial steganography backbone and a software-layer security component that protects the embedded message before it is passed into the image embedding engine.

2.8.1 System Overview

At the architectural level, the suite combines a browser-based frontend, a FastAPI backend, a secure payload service, and a dense adversarial steganography model. During encoding, the system accepts a cover image, a plaintext message, and a passphrase. The message is transformed into a protected payload envelope before it is hidden in the image. During decoding, the hidden payload is extracted from the stego image and can only be recovered into plaintext when the correct passphrase is supplied.

The system also introduces a dedicated check workflow. This workflow does not reveal the message and does not require the passphrase, but it determines whether the image contains a valid protected Steg payload. This makes the suite suitable not only for hidden communication, but also for controlled verification of image status within the application.

2.8.2 Key Features

The proposed system has the following key features:

Adversarial-network-based image embedding built on a dense architecture selected for its strong payload and image-quality trade-off.

Passphrase-protected message handling, which ensures that hidden content remains protected even if extraction is attempted on the stego image.

A three-part workflow consisting of encode, decode, and check operations for practical software use.

A web-based user interface and backend service architecture that make the system accessible beyond a research-code setting.

Validation-oriented handling of uploads, generated outputs, and hidden-payload detection to improve safety and usability.

2.8.3 Comparison of Existing Systems with Proposed System

Table 2.1 compares the proposed secure software suite with representative existing systems and method families across the most relevant practical dimensions.

Table 2.1: Comparison of Existing Systems and the Proposed Secure Steganography Software Suite

System / Method	Embedding Basis	Security Against Modern Steganalysis	User Accessibility	Payload Protection
OpenStego	Classical spatial-domain hiding	Low	High	Optional encryption support
Steghide	Classical transform-oriented hiding	Moderate to low	Moderate	Password-based extraction
Research adversarial models	Deep adversarial embedding	High at model level	Low	Usually prototype dependent

Proposed Steg suite	Adversarial dense image steganography	Higher than classical methods	High	Passphrase-protected payload recovery

2.9 Chapter Outline

This chapter has reviewed the concepts, methods, systems, and tools relevant to secure image steganography. The discussion showed that conventional methods such as LSB substitution and older transform-domain tools remain useful as historical and practical baselines, but they are less resilient to modern machine-learning-based steganalysis. The chapter also showed that adversarial-network-based steganography offers a stronger direction for secrecy, especially when combined with software-level controls. These observations directly motivate the design of the proposed secure steganography software suite presented in the next chapter.

CHAPTER THREE

SYSTEM ANALYSIS AND DESIGN

3.1 Preamble

This chapter presents the analysis and design of the secure steganography software suite developed in this project. It translates the ideas discussed in the literature into a practical system architecture and explains how the software components, security controls, and evaluation metrics work together.

3.2 Overview of the System

The proposed system is a web-based secure steganography software suite that enables a user to hide a text message inside an image, recover the hidden message using the correct passphrase, and check whether a candidate image contains a valid protected payload. The system is centered on three operational workflows: encode, decode, and check.

From a design perspective, the system combines four major layers. The first is the presentation layer, which provides the user with an accessible browser interface. The second is the application layer, which validates requests and coordinates processing through the backend services. The third is the security layer, which performs passphrase validation, key derivation, and authenticated payload encryption. The fourth is the steganography layer, which embeds and extracts the protected payload using a dense adversarial image steganography model.

3.3 Requirements Analysis

The analysis of the system requirements was guided by the security problem identified in earlier chapters. The system had to do more than hide a message in an image. It also had to provide a secure and usable process around that hidden message.

3.3.1 Functional Requirements

The system shall allow a user to upload a cover image and encode a text message into it.

The system shall require a passphrase during encoding so that the concealed message is protected before image embedding takes place.

The system shall allow a user to upload a stego image and decode the hidden message only when the correct passphrase is supplied.

The system shall provide a check operation that reports whether an image contains a valid protected payload without exposing the message itself.

The system shall allow the encoded image to be downloaded after successful processing.

3.3.2 Non-Functional Requirements

Security: the system shall not store user passphrases in plaintext and shall protect message recovery through authenticated encryption.

Usability: the system shall provide a straightforward web interface that can be used by non-expert users.

Performance: the system shall complete normal encode, decode, and check operations within a practical waiting time for standard images.

Maintainability: the system shall separate interface logic, API logic, secure payload logic, and model invocation logic.

Reliability: the system shall return controlled errors for invalid files, missing passphrases, wrong passphrases, and unsupported payloads.

3.4 Dataset Description

The image data associated with this project is based on the DIV2K and MSCOCO datasets. DIV2K provides high-resolution natural images that are useful for studying image quality and fine-detail preservation. MSCOCO provides a broader variety of natural scenes, objects, and textures, making it suitable for evaluating behaviour across diverse real-world image content.

These datasets are important because adversarial image steganography depends heavily on the statistics of natural images. A model trained or evaluated on richer and more varied image content is more likely to generalize to realistic use cases than one trained on narrow or repetitive image sets.

3.5 Physical Design

3.5.1 System Architecture

Figure 3.1 presents the physical architecture of the proposed suite.

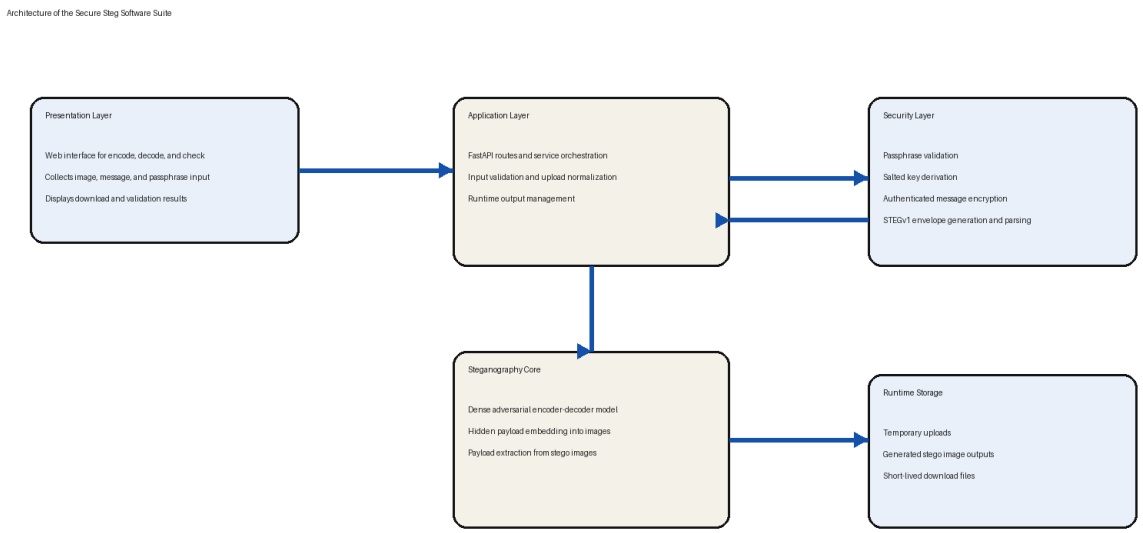

Figure 3.1: Architecture of the proposed secure steganography software suite

The presentation layer is responsible for collecting input and displaying results. The application layer handles routing, validation, and request coordination. The security layer transforms a plaintext message into a protected payload envelope using a passphrase. The steganography core then embeds or extracts that payload through the dense adversarial model, while runtime storage temporarily manages uploads and downloadable outputs.

3.6 Logical Design

3.6.1 Operational Workflow

Figure 3.2 shows the logical workflow of the three main operations of the system.

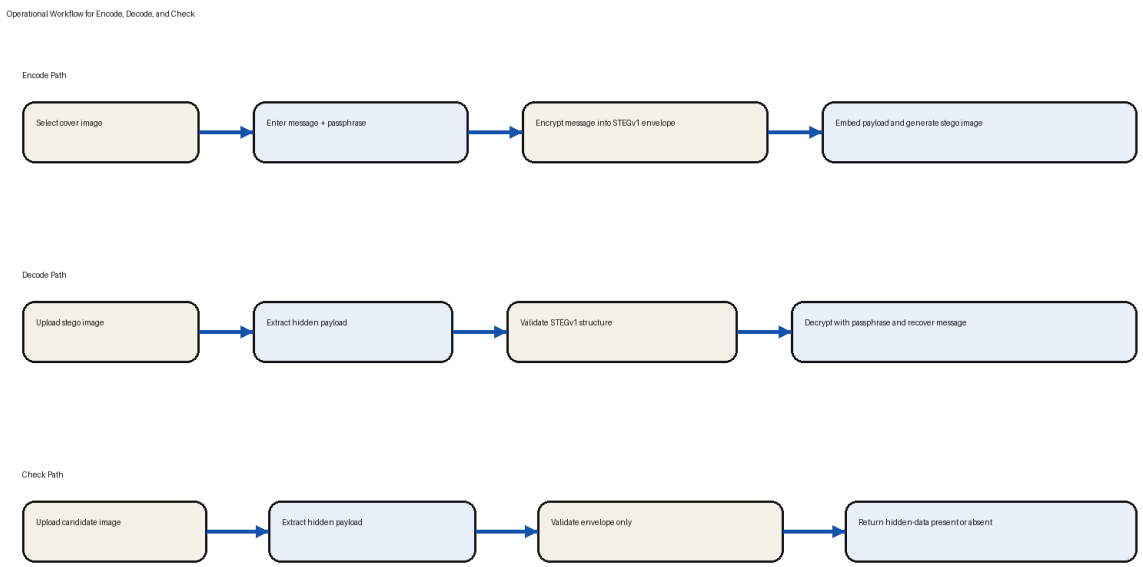

Figure 3.2: Operational workflow for encode, decode, and check

During encoding, the user provides a cover image, a secret message, and a passphrase. The message is first encrypted into a structured payload envelope and then hidden in the cover image. During decoding, the hidden payload is extracted from the stego image and decrypted with the correct passphrase. During checking, the system only tests whether a valid protected payload envelope exists in the image.

3.6.2 Secure Payload Design

A major part of the logical design is the separation between message protection and image embedding. Before the message is hidden in the image, it is transformed into a protected envelope with a version marker, salt, nonce, and authenticated ciphertext. This means that even if a hidden payload is successfully extracted from the image, the message content still requires the correct passphrase before it can be interpreted.

The suite therefore operates as a layered security system. Steganography conceals the presence of the message, while passphrase-based authenticated encryption protects the meaning of the message.

3.7 Mathematical Formulation and Evaluation Metrics

Let C represent the cover image, S represent the stego image, M represent the plaintext message, and P represent the protected payload. The secure payload generation step can be represented as:

$$P = Enc_k(M), \text{ where } k = KDF(passphrase, salt)$$

The image embedding process is then represented as:

$$S = E(C, P)$$

During recovery, the extracted payload P' is passed through the secure decoder:

$$M' = Dec_k(P')$$

To evaluate image distortion, the Mean Squared Error (MSE) between the cover and stego image is given by:

$$MSE = (1 / MN) \Sigma \Sigma [C(i,j) - S(i,j)]^2$$

The Peak Signal-to-Noise Ratio (PSNR) is then computed as:

$$PSNR = 10 \log10(MAX^2 / MSE)$$

Structural Similarity Index Measure (SSIM) is used to evaluate perceptual similarity between the cover and stego images:

$$SSIM(x,y) = ((2\mu x\mu y + C1)(2\sigma xy + C2)) / ((\mu x^2 + \mu y^2 + C1)(\sigma x^2 + \sigma y^2 + C2))$$

For the adopted adversarial method, effective embedding capacity is discussed in terms of Reed-Solomon bits per pixel (RS-BPP), given by:

$$RS\text{-}BPP = D(1 - 2p)$$

where D is the attempted payload depth and p is the bit error rate. These metrics together provide a balanced basis for analysing image quality, payload efficiency, and practical recoverability.

3.8 Summary

This chapter has presented the system analysis and design of the proposed secure steganography software suite. It described the requirements, architecture, workflow, secure payload design, and

evaluation formulas that guide the implementation. The next chapter discusses the actual implementation and evaluation of the system.

CHAPTER FOUR

SYSTEM IMPLEMENTATION AND EVALUATION

4.1 Preamble

This chapter explains how the proposed secure steganography software suite was implemented and how its performance was evaluated. The discussion covers the tools used, the software modules created, the datasets referenced for the adopted adversarial method, and the practical and comparative evaluation of the completed suite.

4.2 System Requirements

4.2.1 Hardware Requirements

A modern personal computer with at least a multi-core processor, 8 GB RAM, and sufficient local storage is adequate for running the software suite with pretrained models. For model training or extended experimentation, a machine with GPU acceleration is preferable.

4.2.2 Software Requirements

The implementation requires Python, PyTorch, FastAPI, Jinja2, Pillow, python-multipart, cryptography, and standard web browser support. During development and deployment, Uvicorn and Gunicorn were used for application serving.

4.3 Implementation Tools

Python was used as the principal implementation language because it provides direct access to the adopted steganography model, deep learning tooling, and backend service development. PyTorch supports the adversarial image steganography backbone. FastAPI was selected for the backend API because it provides a lightweight and structured web service layer. The frontend interface was implemented with HTML, CSS, and JavaScript to provide direct encode, decode, and check interactions. The cryptography library was used to implement authenticated payload protection.

4.4 Program Modules and Interfaces

4.4.1 Encode Module

The encode module accepts the cover image, plaintext message, and passphrase. It validates the inputs, encrypts the message into the STEGv1 payload format, and invokes the adversarial image

encoder to generate the final stego image. The output image is then made available for download through the interface.

4.4.2 Decode Module

The decode module accepts the stego image and the passphrase. It extracts the hidden payload from the image, verifies that it conforms to the supported protected format, and decrypts the payload only when the correct passphrase is supplied. Incorrect passphrases do not reveal the message and instead produce a controlled error.

4.4.3 Check Module

The check module is designed for payload presence verification. It inspects the extracted hidden content and determines whether the image contains a valid protected payload envelope without requiring the passphrase. This module is useful where payload presence must be validated without revealing message content.

4.4.4 Security Module

The security module enforces a minimum passphrase length, derives a key from the passphrase using a salted key derivation function, and uses authenticated encryption to protect the plaintext message before it is hidden in the image. This module is central to the security claims of the software suite because it ensures that concealment and confidentiality operate together.

4.4.5 Frontend and API Interface

The frontend presents the three core workflows in a structured single-page interface. The backend exposes routes for encoding, decoding, checking, and downloading generated outputs. This division makes the application easier to maintain and test.

4.5 Dataset Preparation and Experimental Setup

The project references the DIV2K and MSCOCO datasets as the image datasets associated with the adopted adversarial image steganography approach. DIV2K contributes high-resolution image content, while MSCOCO contributes a wide variety of natural scenes and object-rich images. Together they provide a strong basis for analysing image quality and embedding behaviour across different image characteristics.

The evaluation in this report is considered at two levels. The first level is model-level evidence for the dense adversarial steganography backbone adopted by the suite. The second level is

system-level verification of the secure software suite itself, including functional behaviour, passphrase protection, and hidden-payload detection.

4.6 Evaluation of the System

4.6.1 Functional Validation

Table 4.1 summarises the principal functional checks carried out on the completed software suite.

Operation	Expected Behaviour	Observed Result
Encode	Generate a downloadable stego image from image, message, and passphrase	Successful
Decode with correct passphrase	Recover the original hidden message	Successful
Decode with wrong passphrase	Reject recovery without exposing plaintext	Successful
Check image	Report presence of supported protected payload without passphrase	Successful
Automated webapp tests	Verify API, UI, and service behaviour	47 tests passed

Table 4.1: Functional validation of the secure steganography software suite

The functional validation confirmed that the suite performs the three intended operations correctly. In particular, the passphrase-protected decode workflow ensures that extraction of the hidden envelope alone does not expose the plaintext message.

4.6.2 Security Evaluation

The security evaluation focused on the software-layer protections added to the suite. The system enforces a minimum passphrase length of twelve characters, rejects blank or omitted passphrases, validates the structure of protected payload envelopes, and prevents malformed payloads from being accepted as valid hidden data. These checks reduce the chance of insecure usage and improve the trustworthiness of the check workflow.

4.6.3 Adopted Adversarial Model Performance

At the model level, the suite uses the dense adversarial architecture as its operational default because it offers the strongest overall trade-off among the reported variants. Table 4.2 presents

selected dense-model results reported for the adopted adversarial method on DIV2K and MSCOCO.

Dataset	Data Depth (D)	Accuracy	RS-BPP	PSNR (dB)	SSIM
DIV2K	4	0.82	2.53	37.49	0.88
DIV2K	6	0.70	2.44	38.94	0.90
MSCOCO	4	0.95	3.61	36.94	0.92
MSCOCO	6	0.87	4.40	36.33	0.88

Table 4.2: Selected dense adversarial steganography results adapted from Zhang et al. (2019)

These figures justify the selection of the dense architecture as the software default. They show that the adopted method can maintain acceptable visual quality while supporting a payload level that is substantially higher than what is commonly associated with older practical tools.

4.6.4 Comparison with Traditional Methods

The developed suite was also compared conceptually and practically against weaker traditional approaches. Table 4.3 summarises this comparison.

Criterion	LSB-Based Methods	Transform-Domain Methods	Proposed Steg Suite
Embedding strategy	Fixed pixel-level substitution	Coefficient-domain modification	Learned adversarial dense embedding
Resistance to modern steganalysis	Low	Moderate	Higher
Payload protection	Usually external or optional	Sometimes password-based	Integrated passphrase-protected payload
User workflow	Often simple	Moderate	Encode, decode, and check in one suite
Practical detectability	High under strong detectors	Lower than LSB but still vulnerable	More resilient than classical methods

Table 4.3: Comparison of the proposed suite with traditional steganography approaches

The comparison shows that classical methods remain useful as educational and baseline techniques, but they do not provide the same combination of adaptive embedding, structured security, and practical workflow integration achieved by the proposed suite.

4.6.5 Discussion

The results indicate that the most important contribution of the project is not merely the use of an adversarial embedding backbone, but the translation of that backbone into a secure and practical software system. The suite strengthens hidden communication at the software level by enforcing passphrase-based recovery and by separating encode, decode, and check responsibilities clearly. This makes the project more useful in practice than a model-only demonstration.

The evaluation also shows that the adversarial dense approach is the right backbone for this system when compared with less effective traditional methods. In terms of secrecy and modern detectability, it is better suited to present-day threat conditions than simple LSB or older transform-domain tools.

4.7 Summary

This chapter has presented the implementation and evaluation of the secure steganography software suite. It described the tools used, the system modules, the role of the datasets, the validation of the completed software, and the comparative position of the adopted adversarial method relative to conventional techniques.

CHAPTER FIVE

SUMMARY, RECOMMENDATIONS AND CONCLUSION

5.1 Summary

This project set out to design, implement, and evaluate a secure steganography software suite that is more suitable for modern threat conditions than conventional public steganography tools. The study showed that the growth of machine-learning-based steganalysis has reduced the practical security of many classical methods, especially those based on fixed spatial or transform-domain rules.

To address this problem, the project developed a software suite built around an adversarial-network-based image steganography backbone and strengthened it with a passphrase-protected payload workflow. The resulting system supports image encoding, secure decoding, and hidden-data checking through an accessible interface. The report also compared the adopted adversarial method with weaker traditional methods and showed why the chosen approach is more appropriate for present-day secure hidden communication.

5.2 Recommendations

The following recommendations arise from the study:

Future work should evaluate the suite against additional modern steganalysis detectors under a broader range of practical attack conditions.

The software can be extended to support more carrier media such as audio and video while retaining the same secure payload philosophy.

Further user studies should be carried out to evaluate ease of use, error rates, and adoption by non-expert users.

A future research version of the system may explore stronger benchmarking, model retraining pipelines, and automated reporting of image-quality metrics directly inside the interface.

5.3 Conclusion

In conclusion, this project has demonstrated that a secure and practical steganography software suite can be developed by combining adversarial-network-based image embedding with software-level security controls. The work does not stop at hidden-image generation alone, but

provides a usable secure workflow in which message concealment, controlled recovery, and payload verification are integrated into one system.

The final outcome is an original secure steganography software suite that is academically grounded, practically useful, and more aligned with modern steganalysis-aware security needs than conventional approaches. This makes the study a meaningful contribution to applied information hiding research and a suitable foundation for future publication-oriented refinement.

REFERENCES